IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received July 29, 2018, accepted September 3, 2018, date of publication September 6, 2018, date of current version October 8, 2018.

Digital Object Identifier 10.1109/ACCESS.2018.2868993

A Novel Intrusion Detection Model for a Massive Network Using Convolutional Neural Networks

KEHE WU, ZUGE CHEN, AND WEI LI
School of Control and Computer Engineering, North China Electric Power University, Beijing 102206, China
Corresponding author: Zuge Chen (chenzuge@163.com)

This work was supported in part by the Fundamental Research Funds for the Central Universities under Grant 2017XS073 and in part by the Science and Technology Project of the State Grid Corporation of China under Grant 52100218000U.

ABSTRACT More and more network traffic data have brought great challenge to traditional intrusion detection system. The detection performance is tightly related to selected features and classifiers, but traditional feature selection algorithms and classification algorithms can't perform well in massive data environment. Also the raw traffic data are imbalanced, which has a serious impact on the classification results. In this paper, we propose a novel network intrusion detection model utilizing convolutional neural networks (CNNs). We use CNN to select traffic features from raw data set automatically, and we set the cost function weight coefficient of each class based on its numbers to solve the imbalanced data set problem. The model not only reduces the false alarm rate (FAR) but also improves the accuracy of the class with small numbers. To reduce the calculation cost further, we convert the raw traffic vector format into image format. We use the standard NSL-KDD data set to evaluate the performance of the proposed CNN model. The experimental results show that the accuracy, FAR, and calculation cost of the proposed model perform better than traditional standard algorithms. It is an effective and reliable solution for the intrusion detection of a massive network.

INDEX TERMS Network intrusion detection, convolutional neural networks, image data format conversion, cost function weight, imbalanced dataset.

I. INTRODUCTION

The Internet has become an indispensable part of daily life. Meanwhile, the different types of cyber-attacks are quite harmful to information safety. Therefore, network security has become the focus of many researchers. There are several measures for protecting network security, such as firewalls, encryption, authentication, and intrusion detection. Intrusion detection can identify the illegal behaviors of these attacks through an assumption [1]. It plays an important role in network safety compared with other protective measures, as it can actively detect attacks from network traffic. Generally, there are four attacks: DoS (denial of service), probe, U2R (user to root), and R2L (remote to local) . The goal of intrusion detection is to classify the network traffic into five types: normal, DoS, probe, U2R and R2L. When the traffic is classified into an abnormal attack, the maintainers would take measures to protect network accordingly.

There have been many prior studies on the issue of intrusion detection. These studies generally include two steps: preprocessing and classification [2]. The preprocessing technique, also known as feature selection, is a key process in intrusion detection. It can select some main features from raw data, which have a great impact on results. And it can reduce the size of data storage and improve the training efficiency of model and the accuracy of classifier [3]. It performs well in small input-scale algorithms, though the raw data is high dimensional. Based on whether it is independent of classifier, feature selection can be divided into two methods: filter and wrapper [4]. The filter method is independent of classifier. It selects features according to the statistical characteristic of all raw data. Shi *et al.* [5] proposed a valid filter method for network traffic classification. They used Wavelet Leaders Multifractal Formalism to extract multifractal features, then used PCA-based FS method to remove the irrelevant and redundant features, the results demonstrated significant improvement in accuracy comparing to existing ML-based approaches. And the wrapper method selects features according to the accuracy performance of classifier. As a result, the wrapper method would require more training time and provide better results. However, the classification accuracy

of the feature subset selected by wrappers is strictly depended on the specific classifier, and it is hard to ensure performance with other classifiers [6]. After preprocessing, it needs to have a classifier in traditional intrusion detection. And there have been many machine learning methods are used as the network traffic classifiers, such as SVM (support vector machine) [7]–[10], DTs (decision trees) [11]–[13], ANNs (artificial neural networks) [14], [15], naive Bayesian [16]–[19], fuzzy logic [20]–[23], GAs (generic algorithms) [24], [25], and KNN (k-nearest neighbor) [26], [27]. Also there is hybrid model applied to traffic classification [28]. Ma *et al.* [29] proposed a novel variational Bayesian learning method for the Dirichlet process mixture of the inverted Dirichlet distributions, they would choose the most suitable model for a class from multiple classification models. Also, we can learn that the performances of traditional intrusion detection classifiers are mainly based on the selected features. Additionally, they are difficult to adapt to the massive network, which would generate large amounts of traffic data one day. With the continuous and rapid growth of network traffic data, the machine learning classifiers would not only reduce the accuracy of intrusion detection but also increase the calculation cost and time. While the detection time and detection accuracy are the most significant metrics of intrusion detection system. Only when the system spends little time to gain high accuracy can it ensure the safety of network. Therefore, we hope to put forth a deep learning model to detect intrusion of massive network efficiently and accurately.

Since Lecun *et al.* [30] proposed deep learning (DL), it has been widely applied in visual recognition, speech recognition and natural language processing (NLP). As DL can abstract high level features deeply from raw data, many fields have actively utilized DL algorithms. Additionally, some DL algorithms have been used for intrusion detection. Salama *et al.* [31] first introduced a DBN (deep belief network) into intrusion detection; a DNN was used to reduce feature sets, and SVM classified the traffic. Then, Fiore *et al.* [32] used a discriminative restricted Boltzmann machine to achieve the knowledge of incomplete data, and the model showed that it could successfully adapt to different network environments. Kim *et al.* [33] and Le *et al.* [34] achieved a good result through using long short-term memory (LSTM), which is a type of recurrent neural network (RNN). Additionally, Tang *et al.* [35] applied a deep neural network (DNN) to a software-defined networking (SDN) environment and confirmed that the DL still had strong potential for intrusion detection.

Convolutional neural networks (CNNs) is a classical DL algorithm, and it has been applied in many fields. It can not only select features but also classify the traffic data. Also it can learn better features automatically than traditional feature selection algorithms. The more traffic data, the more useful features the CNN can learn, the better classification the CNN performs, however, machine learning algorithms are easy to over-fit in massive data environment. Hence, CNN is suitable for the massive network environment. Besides, compared with other DL algorithms, the greatest advantage of CNN is that it shares the same convolutional kernels, which would reduce the number of parameters and calculation amount of training once greatly, it can more quickly identify attack type of traffic data. And there have been some studies using CNNs in intrusion detection field [36]–[40]. However, these literatures just only used CNN to select features or classify traffic, and ignored the imbalance of dataset, which is important to detection performance. To resolve imbalanced Internet traffic identification problems, Peng L *et al.* [41] specially proposed an imbalanced data gravitation-based classification method, which performed well. But it needs high computational cost. Liu *et al.* [42] also proposed a undersampling method to imbalance dataset. They reduced numbers of all traffic classes to the number of the class with least number. However, the large reduction of dataset number would reduce feature selection performance of CNN. Therefore, we propose a solution to solve the imbalanced dataset problem in this paper. We improve the CNN based on above solution, and the improved CNN is used for intrusion detection of massive network

The rest of the paper is organized as follows. Section II puts forth the novel intrusion detection model based on improved CNN and introduces every step of the model in detail. It mainly includes data preprocessing, CNN algorithm improvement and evaluation metrics. We design two groups of experiments to select better parameters for the proposed model in section III. Additionally, we conduct some comparisons to evaluate the novel model. Finally, we draw conclusions and future research directions in Section IV.

II. PROPOSED METHODOLOGIES

The novel intrusion detection model based on a CNN includes four steps:

STEP 1: Data preprocessing. This step adjusts the initial data format and normalizes the data values. In order to improve the performance of CNN model, it needs to convert normalized data into image data format.

STEP 2: Training. Training is performed to improve the CNN model performance through constant tuning of the parameters. Additionally, some parameters are modified in the training process, which results in the model achieving better performance.

STEP 3: Testing. Following training (STEP 2), the test data should be used to check the accuracy rate of the CNN model. For example, if the accuracy rate could satisfy the training requirement, the training would cease; otherwise, the model would repeat the training step (STEP 2).

STEP 4: Evaluating. This step is used to evaluate the model performance following training. Generally, evaluation metrics include the accuracy rate, detection rate, and false alarm rate.

Next, Figure 1 shows the four steps of the intrusion detection mode based on the CNN algorithm and the proposed CNN model in this paper:

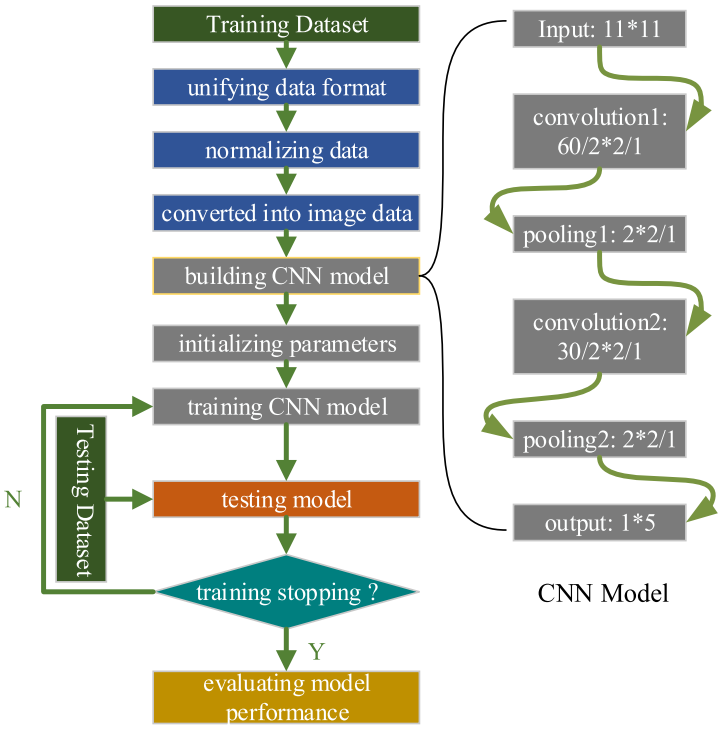

FIGURE 1. Flow chart of the intrusion detection model based CNN.

A. DATASET INTRODUCTION: NSL-KDD DATASET

Tavallaee *et al.* [43] proposed the NSL-KDD dataset selected from the KDDCUP'99 dataset. The KDDCUP'99 dataset was most widely used for detection algorithm evaluation before NSL-KDD. However, the KDDCUP'99 dataset includes a large number of redundant records, which leads to bias towards the more frequent records. Tavallaee *et al.* [43] achieved the NSL-KDD dataset by addressing the KDDCUP'99 dataset. The NSL-KDD dataset has since been considered the benchmark dataset.

Each record of the NSL-KDD dataset is a connection vector that contains 41 features and 1 label. The 41 features can be divided into three groups: 1) basic features; 2) content features; and 3) traffic features, including two smaller groups: ''same host'' features and ''same service'' features. The label marks the attack type of the connection; the attack types include four categories: DoS, probe, U2R, R2L.

The NSL-KDD includes four datasets: KDDTrain+, KDDTrain+_20Percent, KDDTest+, and KDDTest-21. KDDTrain+_20Percent is a 20% subset of the KDDTrain+, while KDDTest-21 is the subset of KDDTest+ that does not contain the records labeled 21. To illustrate the accuracy and universality of the model based on the CNN algorithm, the KDDTrain+, KDDTest+ and KDDTest-21 datasets are used. The distributions of different types of data in the two datasets are shown in Table 1.

TABLE 1. Distributions of different data types.

Datasets / Data Type	KDDTrain+	KDDTest+	KDDTest-21
Normal	67345	9711	2152
DoS	45926	7458	4342
Probe	11655	2421	2402
R2L	995	2754	2754
U2R	52	200	200
Total	125973	22544	11850

B. DATA PREPROCESSING

This paper uses the CNN algorithm to detect attacks, and CNNs have good performance in the field of image processing. Hence, the data preprocessing should first convert the initial data format into image format. Additionally, the NSL-KDD data include two types: symbolic and numeric. The symbolic data should also be mapped to numeric data. Therefore, the data preprocessing includes three steps: 1) symbolic data preprocessing; 2) data normalization; and 3) converting normalized data into image data format.

1) SYMBOLIC DATA PREPROCESSING

The NSL-KDD dataset has four symbolic data types: the protocol_type feature, flag feature, service feature and attack label. In this step, the one-hot encoder is used to map the symbolic data into numeric data. For instance, the protocol_type feature has three categories: TCP, UDP and ICMP, which could be mapped into three three-dimensional binary

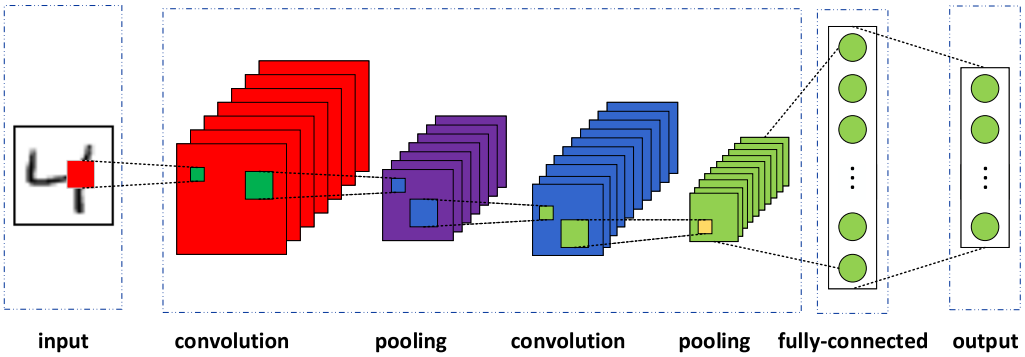

FIGURE 2. The architecture of the convolution neural network.

vectors (1, 0, 0), (0, 1, 0), (0, 0, 1). As a result, one protocol_type feature turns into three features, one flag feature turns into 11 features, and one service feature changes into 70 features. Therefore, the 41 initial features change into 122 features. Notably, the attack label should be classified into five types, NORMAL, DOS, PROBING, R2L and U2R, before using the one-hot encoder.

2) DATA NORMALIZATION

Data normalization can eliminate the differences between different dimensional data and is therefore widely used in computing. As the 122 features of the NSL-KDD dataset have different codomains, in order to ensure the reliability of the training result, we must normalize these features into the range [0, 1]. We apply the min-max normalization to address the data in this paper. The transformation function is as follows:

$$x_{i2} = \frac{x_{i1} - x_{\min}}{x_{\max} - x_{\min}} \tag{1}$$

where x_{i1} represents the initial data i of the feature, $x_{\max}$ represents the maximum data of the feature, $x_{\min}$ represents the minimum data of the feature, and x_{i2} represents the data after normalizing x_{i1}.

3) CONVERTING NORMALIZED DATA INTO IMAGE DATA FORMAT

The initial data format is a 1*122 dimension vector. If the initial data is the input of A CNN model and the image data is the input of B CNN model, when the full connection layers of the two model have equal nodes number, A model has larger calculation amount and lower precision than B model. So we propose the method of converting data format as described in the first step, the feature set includes 122 dimensions. Therefore, we should convert the 1*122 vector into n*n image data. When more actual data are used, the result of the experiment is more accurate, and n should achieve the maximum value. As 121=11*11, the optimal result is "n=11". Therefore, the 122-dimension feature set should remove a feature. When we remove a feature, we usually calculate the correlation or dispersion of features. Such as Ma *et al.* [44] provided a advisable neutral vector variable decorrelation strategies with the serial nonlinear transformation and parallel nonlinear transformation. And we just filter the features preliminarily at this step, we will use CNN to select features further. So the step uses a simple method to select the removed feature, and the method is coefficient of variance (CV), although there are some other feature selection algorithms with better performance. The function of CV is defined as follows:

$$CV = \frac{\sigma}{\mu} \tag{2}$$

where σ is the standard deviation of the feature, μ is the average of the feature, and CV is the CV of the feature. When the feature has a larger CV, it plays a more important role in the feature set. Therefore, we should remove the feature with the minimum CV. However, it is noteworthy that if the average of a feature is equal to 0, the feature should be removed first.

C. METHODOLOGY

Figure 2 shows the architecture of the convolution neural network. The architecture includes five basic components: the input layer, convolution layer, pooling layer, fully connected layer and output layer. An actual CNN model may include several convolution and pooling layers. The detailed introduction for every component is as follows.

Some variables are first defined as follows:

m : m is the number of samples;

n : n is the number of the convolution and pooling layers;

x_i : x_i is the sample i, $i = 1, 2, \cdots, m$;

h_j : h_j is the feature image of layer j, $j = 1, 2, \cdots, n$, and h_0 is the input sample x_i;

w_j : w_j is the convolution kernel of layer j, $j = 1, 2, \cdots, n$;

b_j : b_j is the bias of layer j, $j = 1, 2, \cdots, n$;

y_i : y_i is the output of sample i, $i = 1, 2, \cdots, m$.

Model input sample x_i is the 11*11-dimensional image; then, the image data are inputted into the first convolution layer. The convolution layer is the core of the CNN architecture. As the local perception concept is introduced, all neurons can share the same convolution kernel, and the number

of convolution kernels determines the number of weights. As a result, the number of weights is reduced greatly, and the computational efficiency is increased. The convolution function can be written as:

$$h_j = f(h_{j-1} \otimes w_j + b_j) \tag{3}$$

where $\otimes$ is the convolution function and $f(x)$ is the activation function. This paper uses a nonlinear function——rectified linear unit (ReLU)——as the $f(x)$.

The pooling layer works after the convolution layer. It can reduce the size of feature image h_j and avoid over-fitting. It can be written as:

$$h_j = pool(h_{j-1}) \tag{4}$$

After several convolution and pooling layers, h_j must be reshaped to a vector. Then, output y_i can be achieved through the fully connected layer classifying the vector data. After y_i is achieved, the error between y_i and the expected value needs to be calculated through a loss function. The training aims to minimize the error. In addition, the back propagation of the error uses the gradient descent method generally.

The CNN model proposed in this paper is shown in Figure 1. We can see that the model has two convolution layers and two pooling layers. It has no fully-connected(fc) layer, because we find that the fc layer has no benefit at classification accuracy. Although the convolution kernel size is usually odd number, the paper chooses 2*2 instead of 3*3, because we hope the feature size was integer after pooling layer.

As seen from Table 1, the KDDTrain+ samples distribute unevenly. For example, the ''normal'' data account for more than 50 percent of KDDTrain+. However, the''U2R'' data account for merely 0.04 percent. This leads to a training result bias towards larger data categories. To solve the problem, this paper improves the cost function computation of the CNN.

There are three methods to solve the imbalance problem of samples:

1) SAMPLING-BASED METHOD

The sampling-based method generally includes three methods: oversampling, undersampling, and a hybrid of oversampling and undersampling. The oversampling method increases the number of samples that account for less, whereas the undersampling method reduces the number of samples that account for more. The hybrid of the oversampling and undersampling methods not only increases the number of samples with a smaller proportion but also reduces the number of samples with a higher proportion.

2) ENSEMBLE-BASED METHOD

The ensemble-based method is used to combine classification algorithms with bagging or boosting algorithms. It can improve the performance of the base classifier.

3) COST FUNCTION-BASED METHOD

The cost function-based method adjusts the weights of the cost function according to the proportion of different samples. For example, the samples with smaller proportions have larger weights of the cost function. Conversely, the samples with a higher proportion have smaller weights of the cost function.

The sampling-based methods are easy to implement and work well. However, they change the distribution of the initial dataset, which may lead to increasing new bias. The ensemble-based methods can improve the performance of the model, but they require additional computing resources and training time. However, intrusion detection must detect attacks in real-time, so the ensemble-based methods are not suited to this paper. The third method not only improves the performance of model but also maintains the efficiency of the initial model. Therefore, this paper utilizes the cost function-based method to solve the imbalance problem of samples. The cost function-based method is explained below.

According to characteristics of the NSL-KDD dataset, it can be divided into five categories. Let the number of normal, DoS, probe, U2R, and R2L samples be $n1$, $n2$, $n3$, $n4$, and $n5$, respectively. In addition, maxrepresents the maximum number. In addition, the cost function weights of the sample of category i can be written as:

$$w_i \approx \frac{\max}{n_i} \tag{5}$$

The $\approx$ in (5) shows that w_i is generally calculated using (5), but w_i can be adjusted in a real situation.

D. EVALUATION METRICS

To evaluate the CNN model, this paper selects three indicators: AC (accuracy), DR (detection rate), and FAR (false alarm rate). We first introduce the four parameters TP, FP, FN and TN, which are used to compute AC, DR and FAR. TP (true positive) is the number of attack samples classified into the attack class. FP (false positive) is the number of normal samples classified into the attack class. FN (false negative) is the number of attack samples classified into the normal class. TN (true negative) is the number of normal samples classified into the normal class. We now use Figure 3 to show the relationships among the four parameters.

As a result, AC, DR, and FAR can be written as:

$$AC = \frac{TP + TN}{TP + FN + FP + TN} \tag{6}$$

$$DR = \frac{TP}{TP + FN} \tag{7}$$

$$FAR = \frac{FP}{FP + TN} \tag{8}$$

Hence, the larger AC and DR and the smaller FAR are, the better the performance of the model.

III. EXPERIMENTAL RESULTS AND DISCUSSION

The experiment is performed on TensorFlow [45] using a computer with no GPU acceleration, the operating system is

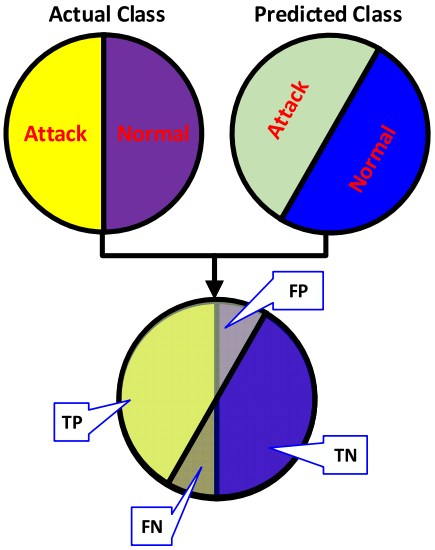

FIGURE 3. The relationships among TP, FP, FN and TN.

Red HAT 4.4.7, and the memory is 8G. The training dataset of the experiment is KDDTrain+, and the testing datasets are KDDTest+ and KDDTest-21. Additionally, 200 epochs and 50 batch sizes are used to train the intrusion detection model. The numbers of the CNN convolution-pooling layers are set as one and two. In this paper, the convolution-pooling layer is defined as the ''multi-hidden layer''.

In section II, we proposed a method of converting normalized data into image data format. In order to prove the method is valid, we design the next experiment based on KDDTest+ dataset: In this experiment there are two CNN model: Init model and Cove model. The input of Init model is 1*122 initial data, and the input of Cove model is the converted 11*11 image data depicted in the Data Processing section. To compare the performance of the two models fairly, the full connection layers of them should have same or similar nodes number. As a result, Init model includes 4 multi-hidden layers. First multi-hidden layer includes 10 convolution kernels of 1*3 dimension, fourth layer includes 20 convolution kernels of 1*3 dimension. The second and third layers include 20 convolution kernels of 1*5 dimension. Conv model includes 2 multi-hidden layers. First layer includes 10 convolution kernels of 2*2 dimension, second layer includes 20 convolution kernels of 2*2 dimension. The strides of all convolutional kernels are 1. And there is a pooling layer after every convolution layer. The pooling kernel of Init model is 1*2 dimension, while the pooling kernel of Conv model is 2*2 dimension. The strides of all pooling kernels are 2. And the full connection layer of Init model has 5=1*5 nodes, that of Conv model has 4=2*2 nodes.

The number of parameters of Init model is:

$$790 = 3*10 + 5*20 + 5*20 + 3*20 + 5*20*5.$$

The calculation amount of Init model is:

$$13680 = 122*3*10 + 60*5*20 + 28*5*20 + 12*3*20 + 5*20*5.$$

TABLE 2. Parameter settings.

layer / parameters	1 multi-hidden layer	2 multi-hidden layers
Convolution layer1-kernel size	6*6	2*2
Convolution layer1-strides	1	1
Pooling layer1-kernel size	2*2	2*2
Pooling layer1-strides	2	2
Convolution layer2-kernel size	-	2*2
Convolution layer2-strides	-	1
Pooling layer2-kernel size	-	2*2
Pooling layer2-strides	-	2

The number of parameters of Conv model is:

$$520 = 2*2*10 + 2*2*20 + 2*2*20*5.$$

The calculation amount of Conv model is:

$$7240 = 11*11*2*2*10 + 5*5*2*2*20 + 2*2*20*5.$$

We can see that Init model needs to spend more time training once than Conv model. Also, the AC of Init model is 78.42%, while AC of Conv model is 79.48%. So we convert the initial format to image data format, which not only improves the precision of model, but also reduces the calculation cost of the model.

To determine the depth of CNN model, we design the next experiment: The input of the CNN algorithm is a converted 11*11 image data, and the dimension of the image is small. Hence, the number of multi-hidden layers is only set as one or two. The parameters with different layers are shown in Table 2.

For the different multi-hidden layers, the experimental results, including AC and test time, are shown in Table 3.

It can be seen from Table 3 that these two models with different layers spend almost the same amount of time testing; nevertheless, the 2 multi-hidden layer model achieves a higher AC. For example, the differences of the two sets of test times are 0.019 s, -0.007 s, and 0.005 s. Meanwhile, the ACs are improved to 1.62%, 1.15%, and 0.83%, respectively. The experimental results show that the 2-layer model with the small and deep convolution kernel has the better performance. Additionally, it spends almost the same time as the 1-layer model does. For massive network intrusion detection, AC and time are of equal importance; AC determines whether attacks can be found, and time determines whether attacks can be defeated. The 2 multi-hidden layer model can meet the real-time requirement of intrusion detection. In conclusion, we choose the 2 multi-hidden layer model for the intrusion detection system. Table 4 shows the confusion matrix of the experiment of 2-layer CNN model on KDDTest+ dataset.

TABLE 3. The experimental results with different multi-hidden layers.

test datasets / multi-hidden layers	KDDTrain+		KDDTest+		KDDTest-21	
	AC	Test Time (s)	AC	Test Time (s)	AC	Test Time (s)
1 layer	0.9784	1.247	0.7833	0.227	0.5988	0.212
2 layers	0.9946	1.228	0.7948	0.234	0.6071	0.207

TABLE 4. Confusion matrix of the experiment of 2-layer CNN model on kddtest+.

Predicted class / actual class	Normal	Dos	Probe	R2L	U2R
Normal	9107	372	217	11	4
Dos	1051	6206	96	103	2
Probe	332	91	1982	14	2
R2L	2005	53	93	597	6
U2R	136	13	16	9	26

Now, there have been many algorithms applied to intrusion detection. These algorithms can be categorized into either machine learning or DL. The machine learning algorithms include J48, naive Bayes, NB tree, random forest, support vector machine, etc. The DL RNN algorithm is also used for intrusion detection. Yin *et al.* [46] calculates the ACs of multiple algorithms used for intrusion detection. The ACs in [46] and the ACs of the CNN model are shown in Figure 4.

It can be seen from Figure 4 that the ACs of the CNN are slightly lower than the ACs of the RNN; the differences between them are 1.81% and 3.96%. However, the ACs of the CNN are higher than the ACs of other the all algorithms. The DL algorithms including RNN and CNN all perform better than all the machine learning algorithms. The result proves that the features learned by DL automatically play more important role than the features selected by traditional feature selecting algorithms. The more network traffic data, the more features the DL algorithm can learn. As a result, DL algorithms are suitable for intrusion detection of massive network.

Since DR and FAR also reflect the performance of a model, this paper computes the DR and FAR of the CNN model. Additionally, Tang *et al.* [35] analyzes the DR and FAR of the RNN used for the KDDTest+ dataset. Table 5 shows the DRs and FARs of CNN and RNN on the whole, and Figure 5 shows the different attack's DRs of CNN and RNN, while Figure 6 shows the different attack's FARs of CNN and RNN.

Figure 5 shows the different attack's DRs of four attacks based on the RNN and CNN. We can see that the DRs of DoS, probe and R2L are higher when using the RNN, but the DR of U2R is lower. In terms of the FAR metric of Figure 6, the FAR of DoS is higher when using the CNN, but the FARs of probe,

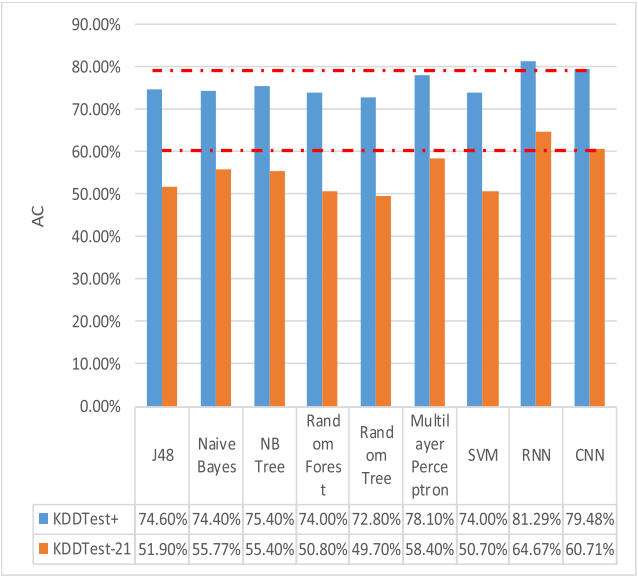

FIGURE 4. ACs of Multiple Algorithms Used for Intrusion Detection.

TABLE 5. DRs and FARs OF CNN and RNN.

models / metrics	CNN	RNN
DR	0.6866	0.6973
FAR	0.2790	0.2689

R2L and U2R are lower. The reason for this is that the cost function weight of the CNN is improved, as introduced in the chapter of methodology. We increase the cost function weights for the classifications with fewer samples based on the set rule, so the CNN model increases their detection rate. The experimental results prove that the method addressing the cost function weight is effective. However, we can find that the AC, DR and FAR are all slightly lower of CNN than RNN from Figure 4 and Table 5.

We can learn that the RNN model of [35] includes one input layer, one hidden layer and one output layer. The input layer has 122 nodes. The hidden layer has 80 nodes. The output layer has 5 nodes. So the number of RNN parameters is:

$$10160 = 122^{*}80 + 80^{*}5.$$

As RNN model is full connection, the calculation amount of it is equal to the parameters number. The CNN model

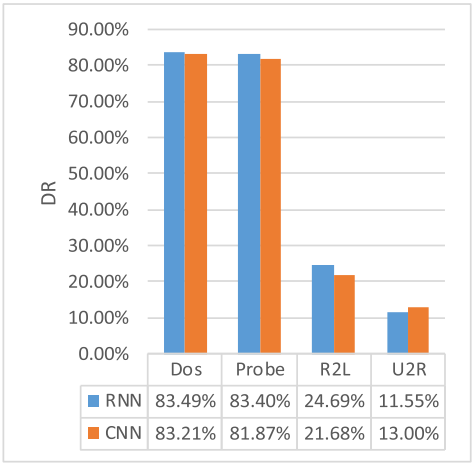

FIGURE 5. Different attack's DRs of the RNN and CNN.

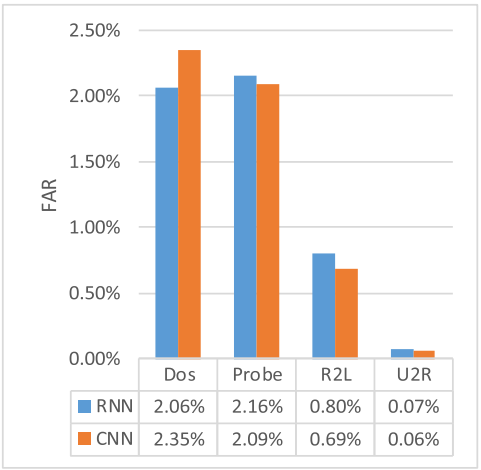

FIGURE 6. Different attack's FARs of the RNN and CNN.

of this paper has fewer parameters number and calculation amount. The number of parameters is:

$$520 = 2^*2^*10 + 2^*2^*20 + 2^*2^*20^*5.$$

The calculation amount is:

$$7240 = 11^*11^*2^*2^*10 + 5^*5^*2^*2^*20 + 2^*2^*20^*5.$$

The parameters number of RNN is almost 20 times that of CNN, and calculation amount of RNN is almost 2 times that of CNN. Hence, the operational efficiency of CNN is 2 times that of RNN, which is meaningful to intrusion detection in real-time. As the metrics of CNN are lower than RNN, we will focus on improving the performance of CNN model in future work.

Li *et al.* [47] also uses CNN for intrusion detection, and it provides a method to convert the data format into image data format. It uses ResNet 50 and GoogLeNet to train the intrusion detection model with the NSL-KDD dataset. The experiment shows that the ACs of KDDTest-21 are greater than 81% with the two algorithms. Table 4 and Table 5 show the experimental results of [47].

TABLE 6. Confusion matrix on KDDTest-21 using ResNet 50.

predicted class / actual class	Normal	Attack
Normal	4	2148
Attack	36	9662

TABLE 7. Confusion matrix on KDDTest-21 using GoogLeNet.

predicted class / actual class	Normal	Attack
Normal	0	2152
Attack	0	9698

It can be seen from Table 6 and Table 7 that the predicted results of normal are mainly incorrect, and the ACs are approximately equal to the detection rate of attack. Especially in Table 7, there is no right predict value of normal traffic data. This shows that there are some problems with the method of converting the data format introduced by [36]. Meanwhile, Table 4 and Figure 5 show that the classify result is normal, which proves that the method of converting the data format presented in this paper is reasonable.

IV. CONCLUSIONS

As DL algorithms can select features from massive data environment automatically and CNN can share weights, a massive network intrusion detection based on CNN is proposed in this paper. To detect attacks in massive network in real time, we propose a method to convert the raw traffic vector format into image data format. We theoretically prove that the image format can reduce the number of computation parameters. In order to improve the detection accuracy of the imbalanced traffic dataset, we propose a method to set the cost function weight coefficients of each class based on its training sample number. The experimental results show that the proposed CNN intrusion detection model performs better that traditional intrusion detection methods. Compared with the RNN model [46], the ACs of the CNN are slightly lower, however, the FAR of CNN are better, also, we theoretically prove that the proposed CNN model needs fewer computation parameters than RNN [46]. It proves that the proposed CNN model is suitable for massive network intrusion detection.

There are still two work to be done in the future. First, we need to improve the detection accuracy. There's plenty of room for performance improvement. We will try to modify the structure of the CNN model to achieve the goal. Besides, as the detection time is also key to intrusion detection, we must ensure that the model can satisfy the intrusion detection time requirement when we improve detection accuracy.

REFERENCES

[1] W. Stallings, *Cryptography and Network Security: Principles and Practice*,vol. 11, no. 7. Englewood Cliffs, NJ, USA: Printice-Hall, 1999, pp. 655–660.

[2] S. Ganapathy, K. Kulothungan, S. Muthurajkumar, M. Vijayalakshmi, P. Yogesh, and A. Kannan, "Intelligent feature selection and classification techniques for intrusion detection in networks: A survey," *EURASIP J. Wireless Commun. Netw.*, vol. 2013, no. 1, pp. 1–16, 2013.

[3] V. Bolón-Canedo, N. Sánchez-Maroño, and A. Alonso-Betanzos, "Feature selection for high-dimensional data," *Prog. Artif. Intell.*, vol. 5, no. 2, pp. 65–75, 2016.

[4] A. L. Blum and P. Langley, "Selection of relevant features and examples in machine learning," in *Proc. AAAI Fall Symp. Relevance*, 1994, pp. 140–144.

[5] H. Shi, H. Li, D. Zhang, C. Cheng, and W. Wu, "Efficient and robust feature extraction and selection for traffic classification," *Comput. Netw. Int. J. Comput. Telecommun. Netw.*, vol. 119, pp. 1–16, Jun. 2017.

[6] J. Shen, J. Xia, X. Zhang, and W. Jia, "Sliding block-based hybrid feature subset selection in network traffic," *IEEE Access*, vol. 5, pp. 18179–18186, 2017.

[7] M. N. Mohammed and N. Sulaiman, "Intrusion detection system based on SVM for WLAN," *Procedia Technol.*, vol. 1, no. 10, pp. 313–317, 2012.

[8] W. Feng, Q. Zhang, G. Hu, and J. X. Huang, "Mining network data for intrusion detection through combining SVMs with ant colony networks," *Future Generat. Comput. Syst.*, vol. 37, pp. 127–140, Jul. 2014.

[9] S. M. H. Bamakan, H. Wang, T. Yingjie, and Y. Shi, "An effective intrusion detection framework based on MCLP/SVM optimized by time-varying chaos particle swarm optimization," *Neurocomputing*, vol. 199, pp. 90–102, Jul. 2016.

[10] H. Wang, J. Gu, and S. Wang, "An effective intrusion detection framework based on SVM with feature augmentation," *Knowl.-Based Syst.*, vol. 136, pp. 130–139, Nov. 2017.

[11] S. S. S. Sindhu, S. Geetha, and A. Kannan, "Decision tree based light weight intrusion detection using a wrapper approach," *Expert Syst. Appl.*, vol. 39, no. 1, pp. 129–141, 2012.

[12] G. Kim, S. Lee, and S. Kim, "A novel hybrid intrusion detection method integrating anomaly detection with misuse detection," *Expert Syst. Appl.*, vol. 41, no. 4, pp. 1690–1700, 2014.

[13] A. S. Eesa, Z. Orman, and A. M. A. Brifcani, "A novel feature-selection approach based on the cuttlefish optimization algorithm for intrusion detection systems," *Expert Syst. Appl.*, vol. 42, no. 5, pp. 2670–2679, 2015.

[14] G. Wang, J. Hao, J. Ma, and L. Huang, "A new approach to intrusion detection using artificial neural networks and fuzzy clustering," *Expert Syst. Appl.*, vol. 37, no. 9, pp. 6225–6232, 2010.

[15] A. Sharma, I. Manzoor, and N. Kumar, "A feature reduced intrusion detection system using ANN classifier," *Expert Syst. Appl.*, vol. 88, pp. 249–257, Dec. 2017.

[16] L. Xiao, Y. Chen, and C. K. Chang, "Bayesian model averaging of Bayesian network classifiers for intrusion detection," in *Proc. Comput. Softw. Appl. Conf. Workshops*, 2014, pp. 128–133.

[17] S. Mukherjee and N. Sharma, "Intrusion detection using Naïve Bayes classifier with feature reduction," *Procedia Technol.*, vol. 4, no. 11, pp. 119–128, 2012.

[18] L. Koc, T. A. Mazzuchi, and S. Sarkani, "A network intrusion detection system based on a Hidden Naïve Bayes multiclass classifier," *Expert Syst. Appl.*, vol. 39, no. 18, pp. 13492–13500, 2012.

[19] P. Louvieris, N. Clewley, and X. Liu, "Effects-based feature identification for network intrusion detection," *Neurocomputing*, vol. 121, no. 18, pp. 265–273, 2013.

[20] S. Elhag, A. Fernández, A. Bawakid, S. Alshomrani, and F. Herrera, "On the combination of genetic fuzzy systems and pairwise learning for improving detection rates on intrusion detection systems," *Expert Syst. Appl.*, vol. 42, no. 1, pp. 193–202, 2015.

[21] P. R. K. Varma, S. S. Kumari, and V. V. Kumar, "Feature selection using relative fuzzy entropy and ant colony optimization applied to real-time intrusion detection system," *Procedia Comput. Sci.*, vol. 85, pp. 503–510, Dec. 2016.

[22] A. Chaudhary, V. N. Tiwari, and A. Kumar, "Analysis of fuzzy logic based intrusion detection systems in mobile ad hoc networks," in *Proc. Elect. Perform. Electron. Packag.*, 2014, pp. 690–696.

[23] A. H. Ali, S. Shamshirband, N. B. Anuar, and D. Petković, "DFCL: Dynamic fuzzy logic controller for intrusion detection," *Facta Univ.*, vol. 12, no. 2, pp. 183–193, 2014.

[24] M. S. Hoque, M. A. Mukit, M. A. N. Bikas, and M. S. Hoque, "An implementation of intrusion detection system using genetic algorithm," *Int. J. Netw. Secur. Appl.*, vol. 4, no. 2, pp. 109–120, 2012.

[25] F. Kuang, W. Xu, and S. Zhang, "A novel hybrid KPCA and SVM with GA model for intrusion detection," *Appl. Soft Comput.*, vol. 18, pp. 178–184, May 2014.

[26] A. A. Aburomman and M. B. I. Reaz, "A novel SVM-kNN-PSO ensemble method for intrusion detection system," *Appl. Soft Comput.*, vol. 38, pp. 360–372, Jan. 2016.

[27] W. Li, P. Yi, Y. Wu, L. Pan, and J. Li, "A new intrusion detection system based on KNN classification algorithm in wireless sensor network," *J. Elect. Comput. Eng.*, vol. 2014, no. 5, pp. 1–8, 2014.

[28] C.-F. Tsai, Y.-F. Hsu, C.-Y. Lin, and W.-Y. Lin, "Intrusion detection by machine learning: A review," *Expert Syst. Appl.*, vol. 36, no. 10, pp. 11994–12000, 2009.

[29] Z. Ma, Y. Lai, W. B. Kleijn, Y. Song, L. Wang and J. Guo, "Variational Bayesian learning for Dirichlet process mixture of inverted dirichlet distributions in non-Gaussian image feature modeling," *IEEE Trans. Neural Netw. Learn. Syst.*, pp. 1–15, Jul. 2018, doi: 10.1109/TNNLS.2018.2844399.

[30] Y. Lecun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[31] M. A. Salama, H. F. Eid, R. A. Ramadan, A. Darwish, and A. E. Hassanien, "Hybrid intelligent intrusion detection scheme," in *Soft Computing in Industrial Applications*. Berlin, Germany: Springer, 2011, pp. 293–303.

[32] U. Fiore, F. Palmieri, A. Castiglione, and A. De Santis, "Network anomaly detection with the restricted Boltzmann machine," *Neurocomputing*, vol. 122, pp. 13–23, Dec. 2013.

[33] J. Kim, J. Kim, H. L. T. Thu, and H. Kim, "Long short term memory recurrent neural network classifier for intrusion detection," in *Proc. Int. Conf. Platform Service*, 2016, pp. 1–5.

[34] T. T. H. Le, J. Kim, and H. Kim, "An effective intrusion detection classifier using long short-term memory with gradient descent optimization," in *Proc. Int. Conf. Platform Technol. Service*, 2017, pp. 1–6.

[35] T. A. Tang, L. Mhamdi, D. McLernon, S. A. R. Zaidi, and M. Ghogho, "Deep learning approach for network intrusion detection in software defined networking," in *Proc. Int. Conf. Wireless Netw. Mobile Commun.*, 2016, pp. 258–263.

[36] R. Vinayakumar, K. P. Soman, and P. Poornachandran, "Applying convolutional neural network for network intrusion detection," in *Proc. Int. Conf. Adv. Comput., Commun. Inform.*, 2017, pp. 1222–1228.

[37] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw.*, Jan. 2017, pp. 712–717.

[38] M. Wang and J. Li, "Network intrusion detection model based on convolutional neural network," *J. Inf. Secur. Res.*, vol. 3, pp. 990–994, 2017.

[39] E. Min, J. Long, Q. Liu, J. Cui, and W. Chen, "TR-IDS: Anomaly-based intrusion detection through text-convolutional neural network and random forest," *Secur. Commun. Netw.*, vol. 2018, Jul. 2018, Art. no. 4943509, doi: 10.1155/2018/4943509.

[40] W. Wang *et al.*, "HAST-IDS: Learning hierarchical spatial-temporal features using deep neural networks to improve intrusion detection," *IEEE Access*, vol. 6, pp. 1792–1806, 2018.

[41] L. Peng, H. Zhang, Y. Chen, B. Yang, "Imbalanced traffic identification using an imbalanced data gravitation-based classification model," *Comput. Commun.*, vol. 102, pp. 177–189, 2016.

[42] Y. Liu, S. Liu, and X. Zhao, "Intrusion detection algorithm based on convolutional neural network," *Beijing Ligong Daxue Xuebao/Trans. Beijing Inst. Technol.*, vol. 37, no. 12, pp. 1271–1275, 2017.

[43] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Int. Conf. Comput. Intell. Secur. Defense Appl.*, 2009, pp. 1–6.

[44] Z. Ma, J.-H. Xue, Z.-H. Tan, Z. Yang, J. Guo, and A. Leijon, "Decorrelation of neutral vector variables: Theory and applications," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 1, pp. 129–143, Jan. 2016.

[45] M. Abadi *et al.* (May 2016). "TensorFlow: A system for large-scale machine learning." [Online]. Available: https://arxiv.org/abs/1605.08695

[46] C. Yin, Y. Zhu, J. Fei, and X. He, "A deep learning approach for intrusion detection using recurrent neural networks," *IEEE Access*, vol. 5, pp. 21954–21961, 2017.

[47] Z. Li, Z. Qin, K. Huang, X. Yang, and S. Ye, "Intrusion detection using convolutional neural networks for representation learning," in *Proc. Int. Conf. Neural Inf. Process.*, 2017, pp. 858–866.

KEHE WU received the Ph.D. degree from North China Electric Power University, Beijing, in 2009. He is currently a Professor with North China Electric Power University and the Director of the Chinese Association for Artificial Intelligence and the Beijing Engineering Research Center of Electric Information Technology. His research interests include smart grid software technology, power information security, and deep learning. He is a Committee Member of the China Electric Power Information Standardization Committee and the Professional Electric Power Information Committee, Chinese Society for Electrical Engineering.

ZUGE CHEN received the B.S. degree in software engineering from North China Electric Power University, Beijing, China, in 2014, where she is currently pursuing the Ph.D. degree with the Beijing Engineering Research Center of Electric Information Technology, School of Control and Computer Engineering. Her interests focus on deep learning and information security.

WEI LI received the B.S. degree in software engineering from North China Electric Power University, Beijing, China, in 1987, where she is currently a professor with the School of Control and Computer Engineering. Her interests focus on smart grid software technology, network security, and machine learning.